

UNIVERSITÀ DEGLI STUDI DI BERGAMO

Dipartimento di Ingegneria Gestionale, dell'Informazione e della
Produzione

Corso di laurea in INGEGNERIA INFORMATICA

Classe n. L-8

**IkigAI – Un career coach basato
sull'Intelligenza artificiale per un orientamento
personalizzato tramite MCP**

Candidati:

Antonio Valentin Radu

Matricola n. 1085992

Andrei Petrisor

Matricola n. 1085993

Relatore:

Chiar.mo Prof. Mauro Pelucchi

Anno Accademico 2024/2025

Indice

Introduzione	5
I Ricerca	7
II Metodologia Adottata	9
II.0.1 Requisiti del progetto	9
II.1 Ruolo dell'MCP	11
II.1.1 Che cos'è MCP?	11
II.1.2 Perché usare MCP?	11
II.1.3 Panoramica dell'architettura MCP	12
II.1.4 Applicazione dell'MCP al progetto	13
II.2 Punti di forza della metodologia	13
II.2.1 Ruolo dello Scrum nel nostro progetto	14
III Architettura	17
III.1 Architettura logica	17
III.1.1 Responsabilità delle fasi	18
III.2 Architettura dei componenti	19
III.2.1 Il cuore logico: l'Engine e la gestione delle sessioni di coaching	20
III.3 Flussi di dati e interazioni	22
III.3.1 Registrazione e accesso	22
III.3.2 Profilazione	22
III.3.3 Sessione di career coaching	22
III.3.4 Persistenza e recupero	23
III.3.5 Integrazioni esterne	23
III.4 Riflessioni finali sull'architettura	24
IV Implementazione	27
IV.1 Tecnologie adottate e motivazioni delle scelte	27
IV.1.1 Next.js	27
IV.1.2 Firebase Authentication	27

IV.1.3	MongoDB Atlas	27
IV.1.4	MCP	29
IV.1.5	Adzuna	32
IV.1.6	Strumenti di versioning	33
IV.1.7	Librerie	33
IV.2	Panoramica visuale del progetto	35
IV.3	Criticità riscontrate e strategie di risoluzione	39
IV.4	Costi	39
IV.4.1	Differenza GPT-3.5 e GPT-4o mini:	40
IV.4.2	Prospettiva futura	40
V	Conclusioni	41
V.1	Sintesi dei risultati ottenuti	41
V.2	Sviluppi futuri	41
	Bibliografia	45

Introduzione

Viviamo in un'epoca caratterizzata da un'enorme disponibilità di opportunità, accompagnata da un flusso costante di informazioni che ci raggiungono in ogni momento della giornata. In questo contesto, risulta complesso riuscire a restare al passo e, ancor di più, saper riconoscere e valorizzare le opportunità che ciascun individuo possiede o può sviluppare. Riteniamo che la consapevolezza rappresenti la chiave per condurre una vita più ricca e significativa.

Il progetto presentato si ispira al concetto di Ikigai, termine giapponese che può essere tradotto come “ragione di essere” o “scopo nella vita”. L'obiettivo principale consiste nello sviluppo di un chatbot basato su intelligenza artificiale capace di analizzare la personalità dell'utente e di suggerire, in maniera mirata, possibili percorsi e alternative utili all'individuazione del proprio scopo esistenziale.

Il servizio è rivolto a chiunque si sia posto interrogativi quali: “Sto realmente seguendo la strada più adatta a me?” oppure “Quali alternative ho a disposizione in questo momento?”. Per questo motivo, l'agente conversazionale sarà accessibile in maniera continuativa, 24 ore su 24 e 7 giorni su 7, consentendo agli utenti di porre domande e ricevere supporto in qualsiasi momento.

Un elemento distintivo del progetto risiede nell'attenzione alla personalizzazione delle interazioni. A tal fine, il sistema utilizza il protocollo MCP (Model Context Protocol)[7], che permette al modello di intelligenza artificiale (nella versione proprietaria GPT-3.5-turbo) di integrare le informazioni aggiornate fornite dall'utente. Ciò consente la generazione di risposte maggiormente coerenti e personalizzate, rendendo l'esperienza conversazionale più efficace e rilevante.

Il lavoro si articola in diverse sezioni. La prima è dedicata all'attività di ricerca, comprendente l'approccio metodologico, la revisione della letteratura, gli strumenti e le fonti utilizzate, nonché la sintesi delle evidenze raccolte. Segue la descrizione della metodologia adottata, con particolare attenzione al ruolo dell'MCP, ai punti di forza individuati e ai limiti riscontrati. La terza parte illustra l'architettura logica proposta e i relativi flussi di dati e interazioni. Nella sezione successiva sono descritte le implementazioni, dalle

tecnologie selezionate al processo di sviluppo, fino alle criticità incontrate e alle strategie di risoluzione adottate. Il lavoro si conclude con una sintesi dei risultati emersi e con la prospettiva di possibili sviluppi futuri, seguita dalla bibliografia di riferimento.

I. Ricerca

L'analisi si è concentrata su due fronti: da un lato la letteratura scientifica relativa all'impiego di modelli linguistici di grandi dimensioni (LLM) e chatbot per il career coaching[1][2], dall'altro lo studio di soluzioni già disponibili online e rivolte a un pubblico eterogeneo.

Dalla revisione emergono due tendenze principali. Una parte consistente dei contributi è orientata allo sviluppo e all'addestramento dei modelli[3], con l'obiettivo di testarli in contesti sperimentali; un'altra si concentra su applicazioni specifiche, come la revisione del curriculum o la proposta di opportunità lavorative[4]. Tuttavia, la maggior parte dei lavori rimane confinata in un ambito accademico o specialistico, senza sviluppare strumenti immediatamente fruibili da utenti non esperti.

Sono stati consultati i principali database scientifici e analizzati prodotti disponibili sul web, attraverso ricerche mirate e forum specializzati.

Dall'analisi emerge che i sistemi di career coaching basati su AI si appoggiano prevalentemente a moduli e questionari[5] per raccogliere le informazioni dell'utente, che vengono poi usate dal modello in un secondo momento. Questa scelta è particolarmente dovuta al fatto che molti studi si sono concentrati sulla dimostrazione dell'accuratezza dei modelli di predizione, utilizzando algoritmi come KNN, Random Forest o Decision Tree. Per addestrare tali modelli, i ricercatori hanno costruito e utilizzato dataset proprietari, mentre per la "career prediction" i dati raccolti dai questionari vengono trasformati per adattarsi al formato richiesto dal modello, ottenendo così una predizione dei lavori più compatibile con il profilo utente.

Tuttavia, l'uso di dataset e modelli personalizzati ha comportato alcune limitazioni: spesso le applicazioni risultano focalizzate su ambiti professionali ristretti, come il settore informatico, e le risposte standardizzate dei form riducono la possibilità per l'utente di esprimere le proprie preferenze o caratteristiche personali in modo naturale. Al contrario, il nostro progetto si propone di superare questi limiti, raccogliendo risposte testuali libere dall'utente e fornendo un'interazione più dinamica e naturale tramite un chatbot. In questo modo l'attenzione si sposta dalla semplice accuratezza del modello verso la qualità dell'esperienza utente, offrendo maggiore libertà di espressione e un percorso di orientamento più personalizzato.

Non sono stati individuati studi che prevedano l'elaborazione dei dati in tempo reale[6], aspetto che rappresenta una lacuna rilevante. Inoltre, le soluzioni esistenti risultano spesso frammentarie, focalizzate su singole fasi del processo (es. valutazione CV), senza una visione integrata del percorso di orientamento.

Questi elementi hanno guidato la definizione del presente progetto, orientandolo verso l'utilizzo di modelli pre-addestrati e di un'architettura capace di gestire le interazioni dell'utente in tempo reale.

Oltre alla letteratura accademica, abbiamo individuato due soluzioni commerciali particolarmente pertinenti e funzionali: COACH: AI-powered Career Coach[30] e AI Career Coach[31]. Il primo, COACH, è un sistema di career coaching supportato da AI progettato per istituti universitari, programmi di formazione e piattaforme di apprendimento online, con l'obiettivo di scalare la guida professionale, colmare gap di competenze e migliorare i risultati per gli utenti. Il secondo, AI Career Coach, fornisce consigli personalizzati sfruttando l'AI e il profilo dell'utente, offrendo indicazioni su cambi di carriera, ricerca di un nuovo lavoro o sviluppo delle proprie competenze.

Il nostro lavoro può essere visto come un prototipo esplorativo che privilegia la qualità dell'interazione e la facilità d'uso, lasciando aperta la possibilità di integrazioni future con capacità analitiche più avanzate.

II. Metodologia Adottata

Per lo sviluppo del progetto abbiamo adottato un approccio agile, nello specifico utilizzando il framework **Scrum**. Questo approccio è stato scelto per la sua capacità di garantire consegne frequenti di software funzionante e di qualità, mantenendo il focus sulle esigenze del progetto e sulla flessibilità rispetto a eventuali modifiche dei requisiti.

I metodi agili si differenziano dai modelli tradizionali, come il modello a cascata (waterfall), che prevedono uno sviluppo lineare e rigidamente strutturato, in quanto favoriscono un approccio iterativo e incrementale. Ogni iterazione, tipicamente della durata di qualche settimana, rappresenta un piccolo progetto a sé stante, comprensivo di tutte le attività necessarie per rilasciare un incremento funzionale del software: pianificazione, analisi dei requisiti, progettazione, implementazione, test e documentazione.

Lo Scrum, in particolare, promuove la formazione di team piccoli, multifunzionali e auto-organizzati, la pianificazione adattiva e la revisione continua delle priorità. Nel nostro caso, i requisiti del progetto sono stati definiti dal team stesso tramite il metodo **MoSCoW**, che ha permesso di classificare le funzionalità in base alla loro priorità (Must have, Should have, Could have, Won't have), simulando così il ruolo del cliente all'interno del processo di sviluppo.

La comunicazione diretta e in tempo reale, preferibilmente faccia a faccia, è uno dei pilastri della metodologia agile. Il team comprende tutte le figure necessarie per completare il progetto, i programmatori sono coloro che definiscono i requisiti. Questo ha permesso di coordinare le attività in modo chiaro e di adattare rapidamente il lavoro alle priorità definite dal team.

II.0.1. Requisiti del progetto

Come anticipato, nel nostro caso i requisiti del progetto sono stati definiti dal team stesso tramite il metodo **MoSCoW**, che ha permesso di classificare le funzionalità in base alla loro priorità.

Di seguito i requisiti principali identificati:

MUST HAVE

- **Career Match:** due percorsi di raccolta dati per l'utente:
 - Percorso **semplificato:** informazioni di base come competenze, interessi, percorso accademico o semplicemente un CV.
 - Percorso **completo:** tutte le informazioni, inclusa la personalità tramite domande mirate sui concetti dell'Ikigai, per una profilazione più precisa e umana.
- **Integrazione concetto di Ikigai:** analisi di valori personali, passioni e fonti di realizzazione per suggerire percorsi professionali significativi.
- **Career Coach:** guida nella definizione e pianificazione di obiettivi professionali e personali basata su:
 - Orientamento agli obiettivi
 - Pianificazione strategica
 - Crescita professionale e personale
- **Interfaccia Web innovativa:** design moderno e funzionale per migliorare l'esperienza dell'utente.
- **Login:** sistema di autenticazione per profilazione e salvataggio dati.
- **Implementazione protocollo MCP:** gestione delle interazioni e dei dati all'interno del sistema.

SHOULD HAVE

- Coaching approfondito passo per passo; il supporto resta generale e orientativo.
- Rilevamento di eventuali cambiamenti negli interessi e nelle priorità dell'utente.

COULD HAVE

- Aggiornamento automatico dei dati dell'utente dopo un certo periodo.
- Supporto a più lingue per ampliare l'accessibilità.
- Possibilità per gli utenti di fornire feedback sul sistema.

WON'T HAVE

- Coaching mirato.
- Recupero chat e ripresa della sessione dal punto di interruzione

II.1. Ruolo dell'MCP

II.1.1. Che cos'è MCP?

MCP (Model Context Protocol) è un protocollo aperto e universale che standardizza il modo in cui le applicazioni forniscono informazioni contestuali ai grandi modelli di linguaggio (LLM).[7]

In poche parole, bisogna immaginare MCP come una porta USB-C per le applicazioni di intelligenza artificiale. Proprio come USB-C fornisce un modo standardizzato per collegare i dispositivi a varie periferiche e accessori, MCP fornisce un modo standardizzato per collegare i modelli di intelligenza artificiale a diverse fonti di dati e strumenti. MCP consente di creare agenti e flussi di lavoro complessi basati sugli LLM e connette i modelli con il mondo.[8]

II.1.2. Perché usare MCP?

I grandi modelli di linguaggio (LLM) rappresentano strumenti estremamente potenti, ma non sono esenti da limiti strutturali. In primo luogo, la loro conoscenza è legata ai dati utilizzati in fase di addestramento e diventa rapidamente obsoleta: l'aggiornamento di un modello richiede infatti ingenti risorse di calcolo e tempi molto lunghi, rendendo complesso mantenere l'informazione sempre aggiornata. Un'ulteriore limitazione riguarda l'assenza di una conoscenza specialistica di dominio: poiché i modelli vengono addestrati principalmente su dati generali disponibili pubblicamente, essi non possono cogliere in profondità processi interni o informazioni proprietarie di specifiche organizzazioni, come nel caso di cataloghi aziendali, sistemi gestionali o archivi medici. A questo si aggiunge la mancanza di uno standard unificato per l'integrazione con fonti esterne: oggi esistono molte soluzioni diverse (ad esempio RAG, basi di conoscenza locali o ricerche web), ma ciascuna richiede sviluppi personalizzati e spesso complessi.

Per affrontare queste criticità è stato introdotto il Model Context Protocol (MCP), un approccio che mira a standardizzare l'accesso degli LLM a dati e strumenti esterni. MCP offre diversi vantaggi: da un lato mette a disposizione un'ampia gamma di integrazioni già pronte, che spaziano dai database ai sistemi di versionamento del codice, fino a strumenti di produttività; dall'altro favorisce una maggiore flessibilità, poiché consente di sostituire facilmente il modello sottostante (ad esempio passando da GPT a Claude o Gemini) senza dover riprogettare l'intero sistema di integrazione. Inoltre, MCP permette di costruire flussi di lavoro complessi che coinvolgono più strumenti e sorgenti informative, rendendo gli LLM capaci di operare in scenari avanzati, come l'analisi documentale o la generazione di report strutturati.

II.1.3. Panoramica dell'architettura MCP

In MCP sono presenti tre ruoli principali: i **Server**, che forniscono strumenti e accesso ai dati; i **Client**, che fungono da ponte tra i server e l'applicazione di intelligenza artificiale; e gli **Host**, cioè le applicazioni che integrano al loro interno sia un modello linguistico sia uno o più client, come ad esempio Claude Desktop, Cursor o Visual Studio Code; che sono client proprietari, oppure si può optare nello sviluppo del proprio client.

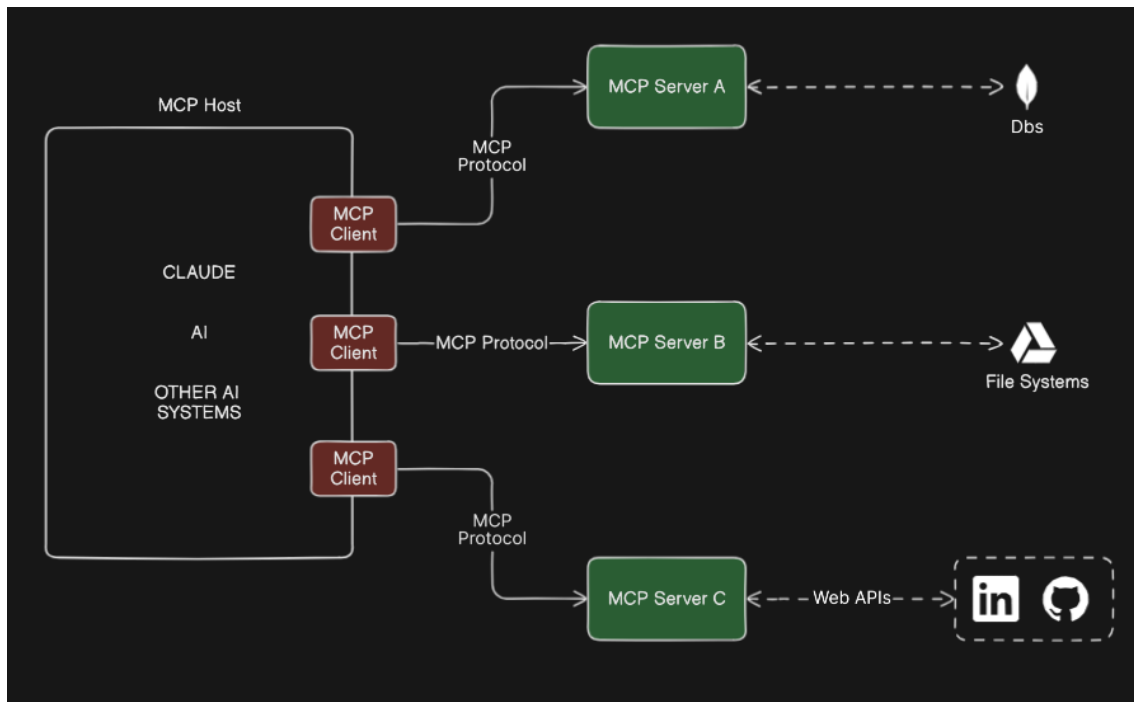


Figura II.1: Vista Architettuale MCP.

Fonte: adattato da ByteByteGo, *Why is everyone talking about MCP?*, YouTube, 2025.[29]

L'architettura segue un modello client-server, in cui ogni host crea un client dedicato per ciascun server con cui stabilisce una connessione. Ciò comporta una relazione uno-a-uno tra client e server. Per esempio, Visual Studio Code può collegarsi contemporaneamente al server di Sentry e a un server locale per il filesystem: in questo caso, il runtime dell'applicazione istanzia due client distinti, ognuno dei quali mantiene la connessione con il proprio server di riferimento. I server, a loro volta, possono essere eseguiti in locale o in remoto: nel primo caso la comunicazione avviene tramite il trasporto STDIO, più efficiente perché non richiede rete; nel secondo caso, invece, viene utilizzato il trasporto HTTP, che consente lo scambio di messaggi anche con sistemi remoti e supporta meccanismi di autenticazione standard come token OAuth o API key.

MCP è organizzato su due livelli concettuali. Il primo è il data layer, che definisce il protocollo di comunicazione basato su **JSON-RPC 2.0**. Questo livello stabilisce la

struttura dei messaggi e regola la gestione del ciclo di vita della connessione, dall'inizializzazione alla terminazione, oltre a introdurre i cosiddetti primitivi, attraverso cui avviene lo scambio di contesto e funzionalità. Il secondo è il transport layer, che gestisce i canali di comunicazione, l'autenticazione e i dettagli legati al trasporto dei dati. Grazie a questa separazione, il livello dei dati rimane indipendente dal meccanismo di comunicazione utilizzato.

Elemento centrale di MCP sono proprio i primitivi, che rappresentano le entità attraverso le quali client e server possono interagire. I server possono mettere a disposizione strumenti eseguibili, come funzioni di interrogazione di database, chiamate API o operazioni su file; possono inoltre esporre risorse, cioè fonti di dati contestuali come schemi, contenuti o risposte, e infine fornire prompt riutilizzabili, ad esempio template di interazione o few-shot per guidare il modello linguistico. Anche i client possono esporre primitivi: tramite il sampling permettono ai server di richiedere completamenti direttamente al modello linguistico dell'host; attraverso l'elicitation consentono di raccogliere ulteriori informazioni o conferme dall'utente; infine, con il logging possono ricevere messaggi di debug e monitoraggio dai server.

Grazie a questa organizzazione, MCP permette agli LLM di integrare dinamicamente strumenti, dati e interazioni fornite dai server, arricchendo al tempo stesso le capacità di dialogo e di azione attraverso la mediazione dei client.

II.1.4. Applicazione dell'MCP al progetto

Come discusso nel Capitolo I, il progetto è stato concepito per operare in tempo reale. Per questo motivo, al modello deve essere resa disponibile l'intera sessione di interazione, inclusa la sequenza di domande relative all'Ikigai. Tale approccio consente di effettuare un Career Match accurato e, soprattutto, di supportare in maniera efficace la fase di Career Coach. In questa seconda fase, infatti, è fondamentale che il modello disponga di dati aggiornati, così da fornire risposte personalizzate che riflettano non solo il profilo iniziale dell'utente, ma anche le sue intenzioni e i suoi obiettivi correnti.

Un ulteriore aspetto rilevante riguarda l'impiego di un server riutilizzabile. Questa soluzione permette non solo di gestire le sessioni in maniera efficiente, ma anche di riutilizzare strumenti, risorse e prompt in contesti applicativi differenti, favorendo così una maggiore flessibilità e scalabilità del sistema.

II.2. Punti di forza della metodologia

Uno dei principali motivi per cui è stato scelto Scrum [9] risiede nei suoi numerosi punti di forza, che lo rendono uno dei framework agili più diffusi e apprezzati nello sviluppo

software.

Scrum favorisce lo sviluppo iterativo e incrementale: al termine di ogni sprint il team è in grado di consegnare un incremento di prodotto funzionante e testato. Questo approccio garantisce la possibilità di validare costantemente le scelte effettuate e di misurare il valore aggiunto delle funzionalità sviluppate, evitando il rischio di arrivare a fine progetto con un prodotto non conforme alle aspettative.

A differenza dei modelli tradizionali, Scrum si distingue per la sua capacità di gestire i cambiamenti di requisiti anche in fasi avanzate del progetto. Ogni sprint planning e backlog refinement permette di aggiornare le priorità in base alle nuove esigenze, assicurando che lo sviluppo rimanga sempre allineato agli obiettivi reali. Questo si è rivelato particolarmente utile nel nostro progetto, dove alcune funzionalità inizialmente considerate secondarie hanno acquisito maggiore rilevanza man mano che il lavoro progrediva.

Scrum pone grande enfasi sulla comunicazione diretta, con eventi strutturati come le daily stand-up, le review e le retrospective. Questi momenti hanno permesso al team di mantenere una visione condivisa del progetto, di individuare rapidamente ostacoli e di migliorare continuamente il proprio modo di lavorare. La natura auto-organizzata del team ha favorito il coinvolgimento attivo di tutti i membri, aumentando motivazione e senso di responsabilità.

L'approccio Scrum integra naturalmente attività di test e revisione durante ogni iterazione, riducendo la probabilità di accumulare errori non gestiti. L'attenzione costante alla qualità si traduce in un software più robusto e manutenibile. Inoltre, la pratica di rilasciare incrementi funzionanti consente di effettuare un monitoraggio continuo delle performance e della stabilità del sistema.

Infine, un ulteriore punto di forza risiede nella flessibilità organizzativa che Scrum concede ai team. Non esiste un modello rigido di gestione, ma piuttosto linee guida che possono essere adattate alle dimensioni del progetto e alle competenze del gruppo di lavoro. Questo ha permesso di modellare il framework sulle reali necessità del nostro progetto, massimizzando l'efficacia delle attività svolte.

II.2.1. Ruolo dello Scrum nel nostro progetto

L'adozione di Scrum non è stata solamente una scelta metodologica astratta, ma ha avuto un impatto concreto sul modo in cui il nostro team ha organizzato e portato avanti il lavoro. Il framework ha fornito una struttura chiara e flessibile, permettendo di gestire le attività in maniera incrementale e di adattarci progressivamente alle necessità emerse durante lo sviluppo.

La prima attività è stata la creazione del product backlog, ovvero l'elenco delle funzionalità da implementare, ordinate in base alla priorità definita con il metodo MoSCoW.

Questa fase ci ha permesso di avere da subito una visione chiara delle funzionalità “essenziali” (Must have) rispetto a quelle “desiderabili” (Should/Could have). A partire da questo backlog principale, ad ogni sprint veniva derivato lo sprint backlog, contenente un sottoinsieme di attività realmente implementabili nel tempo a disposizione.

Il nostro team ha stabilito sprint della durata di una settimana, ciascuno con obiettivi chiari e ben definiti. Durante lo sprint planning, sono state discusse le attività da svolgere e stimati i tempi di realizzazione. Questa fase ha avuto un ruolo chiave nell’organizzazione del lavoro, evitando di sovraccaricare il team e garantendo che le funzionalità sviluppate fossero sempre coerenti con gli obiettivi generali del progetto.

Scrum ha favorito la creazione di un team auto-organizzato: non erano previsti ruoli rigidamente definiti, e ciascun membro ha contribuito a più aspetti del progetto. Le attività hanno spaziato da compiti strettamente tecnici, come lo sviluppo delle componenti di frontend e backend, ad attività di carattere più generale, quali l’analisi delle metodologie da adottare, la definizione delle modalità di implementazione del codice e le fasi di test delle funzionalità sviluppate. Le decisioni sono state assunte collettivamente, in particolare durante i daily stand-up, che hanno consentito al team di condividere lo stato di avanzamento, individuare tempestivamente eventuali ostacoli e riallineare rapidamente le priorità.

Per gestire le attività e monitorare i progressi, sono stati utilizzati strumenti digitali che hanno agevolato la collaborazione e la trasparenza. In particolare, una board Kanban virtuale (Trello) è stata impiegata per visualizzare le user stories in corso, quelle completate e quelle pianificate per gli sprint successivi. L’integrazione con il sistema di versionamento GitHub ha rivestito un ruolo centrale nell’organizzazione del lavoro e nella gestione collaborativa del codice. Oltre alle funzionalità di base per il tracciamento delle modifiche, sono stati sfruttati strumenti avanzati che hanno consentito di adottare un flusso di sviluppo strutturato e in linea con le pratiche professionali. In particolare, le issues hanno permesso di registrare e monitorare in maniera chiara le attività da svolgere, facilitando l’assegnazione dei compiti e la definizione delle priorità. L’utilizzo dei branch ha garantito la possibilità di sviluppare nuove funzionalità o correggere bug senza compromettere la stabilità del codice principale, mentre le pull request hanno costituito un passaggio fondamentale per la revisione e l’integrazione del lavoro svolto. Questo processo ha reso possibile un controllo di qualità continuo e ha favorito la collaborazione tra i membri del team, assicurando un flusso di sviluppo ordinato e coerente.

III. Architettura

III.1. Architettura logica

Per rappresentare al meglio l'architettura si è deciso di creare una **Clean Architecture** del progetto(Fig.III.1), un modello introdotto da Robert C. Martin (Uncle Bob)[10] che mira a separare in modo netto la logica di business dai dettagli implementativi. Questo approccio prevede una stratificazione a cerchi concentrici, dove i livelli interni contengono gli elementi più stabili e indipendenti, mentre quelli esterni racchiudono i componenti più suscettibili a cambiamenti tecnologici, come database, framework e servizi esterni.

Il principio fondamentale della Clean Architecture è che le dipendenze devono sempre puntare verso l'interno: i livelli più esterni conoscono quelli interni, ma non viceversa. In questo modo, le entità del dominio e i casi d'uso rimangono protetti da modifiche legate alle tecnologie utilizzate, rendendo il sistema più manutenibile, testabile e flessibile.

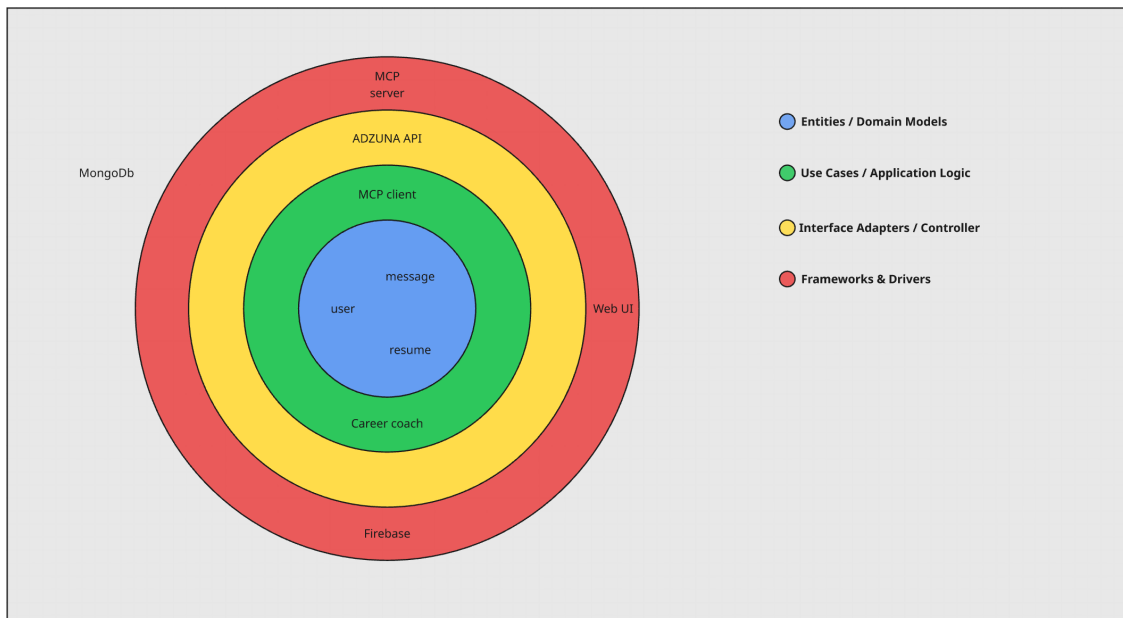


Figura III.1: Clean Architecture
Fonte: elaborazione degli autori

III.1.1. Responsabilità delle fasi

Nel progetto, il livello più interno è costituito dalle entità principali: **user**, **message** e **resume**; che rappresentano le strutture dati fondamentali su cui si basa l'intero sistema. Message rappresenta le interazioni tra l'utente e il career coach all'interno di una sessione. Ogni messaggio include domande e risposte, insieme a informazioni temporali che permettono di ricostruire la conversazione e analizzare il comportamento dell'utente. Resume conserva i documenti caricati dall'utente, come CV o certificazioni, associandoli sia all'utente sia alla specifica sessione in cui sono stati caricati. Queste informazioni consentono al sistema di valutare il profilo professionale e supportare le fasi di orientamento.

Attorno a queste si colloca la logica applicativa, incarnata dal **career coach** (qui uniamo le due fasi spiegate nel Capitolo II.0.1), responsabile dell'elaborazione delle informazioni e dell'orchestrazione dei processi. Dopo la fase iniziale di **profiling**, in cui vengono raccolti i dati dell'utente e analizzato il curriculum, il sistema propone un insieme di possibili percorsi professionali; successivamente, nella fase di **career coaching**, accompagna l'utente con suggerimenti e strategie mirate. Il passaggio da una fase all'altra è gestito attraverso un controllo dello stato dell'interazione, che permette al career coach di attivare le funzioni adeguate in base al contesto e mantenere la coerenza del percorso.

Dal punto di vista tecnico, la comunicazione tra frontend e backend è stata un aspetto importante. L'integrazione di chatbotloop con il client MCP, che si basa sulla SDK di Model Context Protocol e crea un child process, ha infatti generato conflitti con l'ambiente di Next.js, interferendo con i processi principali delle pagine. Per superare questo limite, si è optato per l'introduzione di un livello intermedio basato su API REST, in modo da non importare direttamente la SDK nel frontend. In questo schema, le API fungono da canale di comunicazione: le richieste provenienti dal frontend vengono inoltrate al backend, che elabora le informazioni tramite le funzioni del career coach e restituisce la risposta pronta per essere mostrata all'utente nell'interfaccia. Questa soluzione permette al sistema di mantenere separati i processi legati alla logica da quelli grafici, e semplifica eventuali future estensioni o sostituzioni dei componenti.

Un ulteriore livello è rappresentato dagli adattatori e dai controller, come l'**MCP client**, che si occupano di mediare tra la logica interna e i sistemi esterni. In particolare, l'MCP client permette al career coach di comunicare con il **server MCP**, gestendo lo scambio di informazioni e l'invocazione delle funzioni necessarie. Come descritto nel Capitolo II.1.3 l'architettura del Model Context Protocol prevede la possibilità di collegare più server, ognuno con il proprio ambito di competenza, a un singolo client, così da costruire un ecosistema distribuito e modulare. Nel nostro caso, la configurazione attuale prevede un solo client e un unico server, all'interno del quale sono però integrate diverse funzionalità.

Infine, il livello più esterno è costituito dai framework e dai servizi esterni: **MongoDB**

per la persistenza dei dati, **Firestore** per l'autenticazione e la gestione dei profili, la **Web UI** come interfaccia utente, il **server MCP** (per consultare funzionalità IV.1.4) per la gestione delle comunicazioni e l'**API di Adzuna** per il reperimento degli annunci di lavoro. Questa stratificazione ha permesso di costruire un sistema modulare, dove ogni livello può essere modificato o sostituito senza compromettere l'integrità complessiva della soluzione.

L'adozione della Clean Architecture si è rivelata una scelta particolarmente vantaggiosa per lo sviluppo del progetto. In primo luogo, ha consentito di mantenere una netta separazione tra la logica di business e le tecnologie esterne, evitando che il cuore del sistema fosse vincolato a scelte implementative specifiche. Questo ha reso l'architettura intrinsecamente più flessibile: un eventuale cambiamento del database o l'integrazione di una nuova API esterna non inciderebbero sulle entità interne, preservando così la stabilità delle componenti centrali.

Un ulteriore beneficio riguarda la scalabilità e la manutenibilità. Grazie alla chiara suddivisione dei livelli, ciascun componente può evolvere in maniera indipendente, senza generare effetti indesiderati sugli altri moduli. Ciò significa che il sistema può adattarsi più facilmente a nuove esigenze e crescere nel tempo senza richiedere pesanti interventi di ristrutturazione.

L'approccio adottato ha permesso non solo di migliorare l'organizzazione del codice, ma anche di costruire una base solida e duratura per lo sviluppo futuro, garantendo affidabilità e flessibilità.

III.2. Architettura dei componenti

Oltre alla visione stratificata offerta dalla Clean Architecture, è stata definita una vista dei componenti eseguibili che rende esplicite le dipendenze concrete, i canali di comunicazione e i principali data store. La Figura III.2 illustra i moduli che compongono il sistema e i relativi flussi informativi.

L'interazione dell'utente avviene attraverso la Web Page, che raccoglie l'User input (credenziali, curriculum, preferenze e richieste) e funge da front-end per tutte le funzionalità applicative. La fase di User Signup delega a Firestore l'autenticazione e la gestione dell'identità: la pagina invia i dati di registrazione, Firestore emette i token e ne garantisce la verifica durante le sessioni successive. Una volta autenticato, l'utente viene guidato nella User Profiling, in cui le informazioni riguardanti competenze, interessi, località, seniority e CV vengono normalizzate e inoltrate all'Engine per l'elaborazione.

Completato il profiling, l'utente accede alla fase di Career Coach, strutturata come una conversazione guidata: qui può porre domande e ricevere risposte personalizzate. L'Engine, sfruttando un modello di linguaggio (LLM), fornisce suggerimenti e analisi

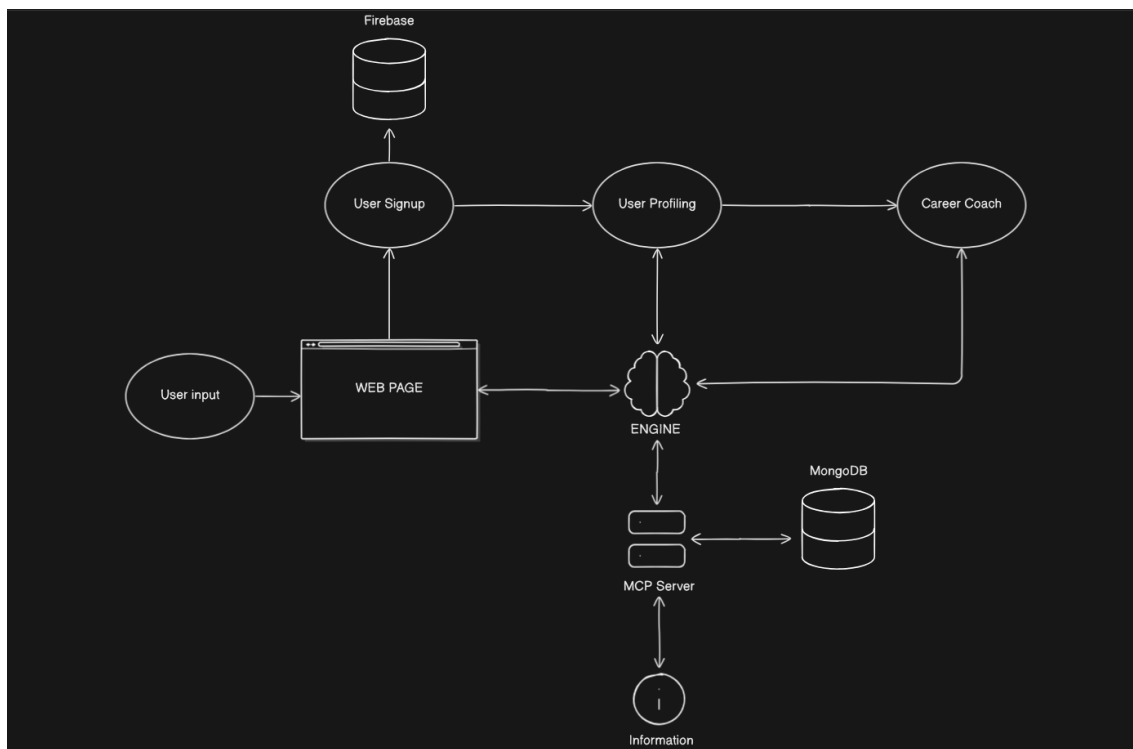


Figura III.2: Component Architecture
 Fonte: elaborazione degli autori

non solo sulla base della conoscenza intrinseca del modello, ma anche considerando il contesto della sessione e i dati precedentemente raccolti durante il profiling. Questo garantisce un'interazione più coerente e adattata al profilo individuale dell'utente.

III.2.1. Il cuore logico: l'Engine e la gestione delle sessioni di coaching

L'Engine rappresenta il cuore pulsante del sistema, poiché concentra al suo interno tutta la logica applicativa che regola il flusso delle interazioni con l'utente e il coordinamento dei vari moduli. Ogni fase del percorso dell'utente, dal profiling iniziale fino al career coaching, viene orchestrata dall'Engine, che riconosce lo stato della conversazione e decide come evolvere il dialogo. La scelta progettuale di strutturare il sistema come un insieme di chat persistenti rafforza l'usabilità: ogni volta che un utente avvia una nuova sessione (sia in modalità "simplified" che "completed"), il flusso ricomincia sempre dal profiling e procede successivamente verso il career coach. In ogni caso, l'utente mantiene la possibilità di riaprire le proprie chat pregresse, consultare lo storico delle interazioni e riprendere la conversazione esattamente da dove era stata interrotta.

Un aspetto fondamentale dell'Engine è l'integrazione con il modello di intelligenza artificiale **GPT 3.5** o **GPT4o-mini**(è possibile usufruire di qualsiasi modello, basta possedere la chiave API), che funge da generatore principale delle risposte.

Abbiamo optato per i modelli GPT in quanto già avevamo familiarità con le API OpenAI, il che ha ridotto la curva di apprendimento e i tempi di integrazione rispetto a soluzioni open-source da configurare e fare fine-tuning manualmente (ad es. LLaMA o Mistral). Inoltre, per utilizzare questi modelli in autonomia sarebbe stato necessario farli girare localmente, con conseguenti requisiti hardware significativi che non erano alla nostra portata. È vero che esistono anche servizi esterni che espongono LLaMA e Mistral tramite API, ma le versioni realmente performanti sono comunque a pagamento; a quel punto, è risultato più conveniente adottare un modello “plug-and-play” come GPT, che ci garantiva sia semplicità d’uso sia qualità delle risposte.

Abbiamo scelto di integrare due modelli per bilanciare al meglio costi e prestazioni: GPT-3.5, più economico e adatto alle richieste generiche, e GPT-4o-mini, leggermente più costoso ma capace di garantire risposte più accurate nei casi che lo richiedono. Un confronto dettagliato dei costi è riportato nel Capitolo IV.4

Questi modelli non si limitano a fornire risposte generiche, ma vengono costantemente arricchiti con le informazioni relative alla sessione corrente dell’utente, permettendo al chatbot di rispondere in maniera più personalizzata e contestualizzata. L’Engine, infatti, salva progressivamente ogni input e ogni dato rilevante su MongoDB tramite i tool(IV.1.4) forniti dall’MCP Server, così che il modello abbia accesso a uno storico aggiornato e coerente della conversazione. In questo modo, l’intelligenza artificiale è in grado di analizzare non solo il singolo messaggio, ma anche l’intero contesto pregresso, producendo risposte più pertinenti e personalizzate.

Questa integrazione risulta particolarmente significativa nella fase di profiling. Durante questo processo, le risposte dell’utente vengono elaborate dall’AI in tempo reale e arricchite da un’analisi che tiene conto sia del contenuto esplicito delle risposte sia dello storico già accumulato nella sessione. Alla fine del profiling, il sistema è in grado di proporre un **career match**, ossia una selezione di profili professionali potenzialmente più adatti all’utente. La forza di questo approccio risiede nel fatto che l’AI non si basa soltanto su un set fisso di regole, ma su una base di conoscenza ampia e dinamica, continuamente aggiornata grazie al salvataggio costante dei dati. Ciò consente al chatbot di affinare progressivamente le sue analisi e di formulare suggerimenti sempre più mirati e realistici rispetto al profilo emerso.

Quindi, l’Engine svolge una funzione duplice: da un lato gestisce lo stato del sistema, garantendo la continuità e la coerenza del dialogo tra profiling e career coach; dall’altro abilita l’intelligenza artificiale ad operare in modo realmente personalizzato, grazie alla persistenza dei dati e all’integrazione con i tool e le risorse offerte dall’MCP Server. Questo equilibrio tra orchestrazione logica e potenziamento tramite AI rende il sistema capace non solo di rispondere in maniera naturale e fluida, ma anche di costruire progressivamente

una conoscenza contestuale dell'utente, trasformando ogni chat in un ambiente interattivo, personalizzato e duraturo.

III.3. Flussi di dati e interazioni

III.3.1. Registrazione e accesso

L'interazione tipica inizia con la registrazione e l'autenticazione dell'utente; per questa operazione è stato scelto **Firebase** [12], una piattaforma di autenticazione gestita che offre vantaggi pratici per un contesto di prototipo/servizio web: permette di integrare con semplicità metodi di accesso standard (email/password, provider social), fornisce token di sessione sicuri verificabili lato server e semplifica la gestione di criteri di sicurezza e regole di accesso. Dopo la registrazione l'utente dispone di un profilo persistente che abilita l'accesso alle funzionalità del sito e che costituisce il punto di partenza per tutti i flussi successivi.

III.3.2. Profilazione

La fase di profilazione è il primo momento in cui l'Engine entra effettivamente in gioco. In questo progetto la profilazione è concepita come una raccolta strutturata di informazioni che combina domande ispirate al concetto di **ikigai**. La profilazione può essere eseguita in modalità completed, un percorso approfondito in cui vengono poste anche le domande "ikigAI" per ottenere quadro più ricco e completo dell'utente, oppure in modalità simplified, in cui si riduce il numero di domande ma si compensa con una forte integrazione dei tool dell'MCP, ad esempio con l'analisi automatica del **CV** e la selezione automatizzata delle opportunità professionali più affini. In entrambi i casi tutte le risposte e i file caricati vengono normalizzati e inviati all'Engine per l'elaborazione.

III.3.3. Sessione di career coaching

Completata la profilazione, la sessione evolve nella fase di **Career Coach**. Qui la struttura conversazionale viene riempita di contenuti: l'utente può scegliere una professione suggerita dal profiling (o una offerta risultata idonea tramite la ricerca per preferenze) e chiedere al sistema di generare un piano personalizzato. Il career coach è progettato per produrre, in pochi secondi (tipicamente nell'ordine di 10–15 secondi per la generazione iniziale), un piano dettagliato fatto su misura per la persona e per la posizione selezionata: il piano include passi operativi, suggerimenti per la modifica del CV, linee guida per il colloquio e azioni di upskilling. È importante evidenziare che, dopo la generazione di questo piano iniziale, la conversazione diventa fortemente interattiva e guidata dall'utente: mentre nella

fase di profilazione era necessario rispondere a domande strutturate per costruire il profilo, nella fase di coaching l'utente assume un ruolo proattivo e può porre domande aperte su qualsiasi punto del piano, approfondire aspetti specifici o richiedere variazioni. Questo comportamento trasforma il modulo career coach in una vera e propria chat dove l'utente è il protagonista delle richieste e il sistema risponde in modo contestualizzato.

A rendere possibili risposte contestuali e personalizzate è l'integrazione con un modello di linguaggio (ChatGPT 3.5), che viene utilizzato per la generazione delle risposte testuali. L'Engine fornisce al modello il contesto della sessione corrente, ossia lo storico della chat, le informazioni ottenute dai tool dell'MCP e i dati relativi alla specifica offerta di lavoro, in modo che il **LLM** possa produrre output non solo coerenti con la sua base di conoscenza generale, ma anche fortemente adattati alla singola esperienza utente. In pratica, ogni nuova risposta dell'utente viene salvata e resa disponibile al modello: questo flusso continuo di salvataggio incrementale permette al sistema di avere una "memoria di sessione" sempre aggiornata, che migliora la pertinenza delle risposte e abilita funzionalità come il career match dinamico e le spiegazioni contestualizzate delle scelte proposte.

III.3.4. Persistenza e recupero

Per quanto concerne la persistenza e la capacità di recupero delle sessioni, tutte le entità long-lived, profili utente(Firebase), messaggi della chat, versioni di curriculum e piani generati, vengono memorizzate in MongoDB. L'Engine espone operazioni di lettura e aggiornamento per preservare la coerenza in presenza di retry o riconnessioni lato client; le credenziali e i token di sessione rimangono invece gestiti da Firebase, che autorizza ogni richiesta. La riapertura di una sessione di coaching è resa disponibile mediante un tool dell'MCP che restituisce lo storico completo della chat: l'utente visualizza l'elenco delle sessioni passate, sceglie quella di interesse e può rientrarvi in qualunque momento. Il recupero avviene con un'unica chiamata controllata verso l'MCP, che carica dal database l'intera cronologia e la normalizza per l'Engine, che la reinserisce nel contesto conversazionale del LLM.

III.3.5. Integrazioni esterne

Le integrazioni con fonti esterne hanno un ruolo importante, perché permettono al modello di accedere a informazioni aggiornate che non ha avuto in fase di addestramento. Un esempio è l'uso di dati su offerte di lavoro in tempo reale, elenchi di competenze richieste dalle aziende o statistiche sulle retribuzioni. Questi elementi arricchiscono l'analisi e rendono i suggerimenti più concreti e vicini al contesto attuale del mercato del lavoro. E' l'MCP Server a interrogare i provider esterni (tra cui l'API di Adzuna(IV.1.5)) e a restituire risultati pronti all'uso. Questo approccio garantisce che le risposte del LLM

siano arricchite con informazioni correnti e pertinenti, favorendo decisioni e suggerimenti aderenti alla situazione reale del mercato del lavoro. Va sottolineato che le offerte ottenute via API esterne sono aggiornate dalla fonte stessa e pertanto riflettono lo stato corrente del mercato; l'Engine le incorpora nel processo di matching per proporre opportunità coerenti con le preferenze espresse dall'utente (luogo, stipendio, tipo di contratto, ecc.).

L'architettura attuale offre ancora ampi margini di miglioramento. I tool oggi disponibili all'interno dell'MCP, come l'analizzatore del CV, la ricerca di offerte di lavoro e l'accesso alle sessioni, rappresentano un primo nucleo funzionale, ma la struttura è pensata per essere facilmente ampliata. In futuro si potrebbero aggiungere connettori verso nuove fonti, ad esempio feed di notizie, API di piattaforme di formazione online, servizi per la verifica delle certificazioni o sistemi statistici per confrontare i dati del mercato del lavoro.

La strategia consigliata è quindi un arricchimento progressivo dell'MCP tramite l'aggiunta di tool specializzati e verificati, supportato dall'introduzione di meccanismi di caching, di validazione dei dati e di metriche per monitorare la qualità dei suggerimenti prodotti.

III.4. Riflessioni finali sull'architettura

L'architettura progettata rappresenta un compromesso equilibrato tra rigore metodologico e esigenze pratiche del progetto. La scelta di adottare la Clean Architecture ha permesso di mantenere una netta separazione tra la logica di business e i servizi esterni, favorendo un disaccoppiamento che rende il sistema più flessibile e manutenibile nel tempo. Questo approccio si è rivelato particolarmente utile in un contesto in cui diverse tecnologie convivono; dal database MongoDB, all'autenticazione con Firebase, fino all'uso del modello LLM e dei tool offerti dal MCP Server, senza che nessuna di esse vincoli o condizioni in modo eccessivo la struttura interna del progetto.

In questo contesto, l'Engine ha avuto un ruolo fondamentale. È il componente che gestisce l'andamento della conversazione e dell'applicazione, passando dalle prime fasi di profilazione alla creazione del piano di career coaching, fino alle interazioni successive con l'utente. Oltre a indirizzare le richieste, l'Engine unisce le risposte del modello linguistico con lo storico della sessione e con i dati forniti dai tool esterni. In questo modo riesce a restituire risposte più coerenti, personalizzate e basate su informazioni sempre aggiornate. Grazie a questo approccio, il chatbot non si limita a generare testi, ma diventa un sistema capace di adattarsi al contesto e di fornire un supporto più ricco e utile.

Dal punto di vista operativo, l'infrastruttura ha mostrato di funzionare in modo stabile e di saper mantenere le informazioni delle sessioni degli utenti. In questo modo chi utilizza il sistema può interrompere la conversazione e riprenderla in un secondo momento, trovando

ancora disponibili i dati precedenti. Per organizzare e salvare questi stati si è scelto di usare MongoDB, mentre la gestione dell'accesso e dell'identificazione degli utenti è stata affidata a Firebase. L'idea alla base di questa scelta è stata quella di appoggiarsi a servizi già affidabili e collaudati per compiti considerati delicati, così da ridurre la possibilità di errori e rendere più semplice il lavoro di manutenzione complessiva.

Le integrazioni esterne hanno rappresentato un ulteriore valore aggiunto. L'impiego di API di terze parti, come Adzuna per l'aggiornamento costante delle offerte di lavoro, ha reso il sistema capace di fornire all'utente non solo risposte statiche, ma anche contenuti aggiornati e contestuali. L'MCP Server, fungendo da mediatore verso tali fonti e concentrando l'accesso a tool e risorse, ha reso possibile arricchire le funzionalità del progetto senza aumentare la complessità interna dell'Engine.

In un'ottica verso il futuro, questa architettura offre ampie possibilità di evoluzione. La sua natura modulare, ispirata ai principi della Clean Architecture, permette infatti di sostituire o potenziare singoli componenti senza dover ripensare l'intero sistema. In pratica, è possibile aggiungere nuovi strumenti MCP, ampliare la base di dati utile al coaching o integrare modelli di linguaggio più avanzati, mantenendo stabile la struttura generale. Un aspetto importante è che il server MCP è stato pensato per essere utilizzabile da chiunque: chi lo desidera può prenderlo, modificarlo secondo le proprie esigenze e arricchirlo con tool personalizzati. Questa caratteristica ne aumenta la riusabilità, perché consente di adattare lo stesso nucleo tecnologico a progetti diversi senza dover ripartire da zero. Puntare su un'architettura riutilizzabile significa ridurre i tempi di sviluppo, favorire la collaborazione e garantire maggiore flessibilità nell'evoluzione futura del sistema. Allo stesso tempo, la gestione a stati tramite sessioni persistenti apre la strada a funzioni di analisi più avanzate, capaci di seguire l'evoluzione del percorso dell'utente e di rendere il supporto sempre più mirato e personalizzato.

Nel complesso, l'architettura del sistema si è dimostrata robusta, scalabile e aperta al futuro, riuscendo a coniugare le esigenze immediate del progetto con una visione di lungo periodo orientata all'espandibilità e alla sostenibilità tecnica.

IV. Implementazione

IV.1. Tecnologie adottate e motivazioni delle scelte

IV.1.1. Next.js [v15.4.5]

Per l'implementazione dell'applicazione Web è stato scelto di utilizzare **Next.js**[11], un framework basato sulla libreria React che permette di sviluppo di siti complessi in maniera efficiente e veloce. L'adozione di questa tecnologia è motivata da due fattori principali:

- Rendering ibrido - Possibilità di generazione di pagine statiche e di rendering lato server per prestazioni migliori.
- Supporto di TypeScript e Tailwind - Aiutano rispettivamente nella robustezza del codice e nella creazione semplificata di UI efficaci.

IV.1.2. Firebase Authentication [v12.0.0]

Per l'autenticazione è stato utilizzato **Firebase Authentication**[12], servizio che permette la registrazione e l'accesso degli utenti tramite **email e password**, con relativa verifica dell'indirizzo, oppure l'accesso semplificato mediante un **account Google**. La motivazione dietro l'utilizzo di questo servizio è sicuramente la **facilità di integrazione** e la semplicità nel salvataggio delle informazioni essenziali di un utente.

IV.1.3. MongoDB Atlas [13] [v6.18.0]

E' un multi-cloud database, utilizzato per il salvataggio dei messaggi e delle sessioni degli utenti.(Rif. utilizzo pratico - III.3.4). Il database è caratterizzato dalla presenza di due collezioni: "**CVs**" e "**Sessions**".

CVs

In questa collezione avviene l'archiviazione di tutti i CV inseriti dagli utenti, mantenendo anche informazioni come lo userID (id), la sessionId (session) e il un timestamp del

caricamento. Oltre ai metadati, il contenuto del curriculum viene archiviato in formato Base64 [27]. Questa è una codifica che converte i dati binari (ad esempio file PDF, DOCX o altri formati) in una sequenza di caratteri testuali (lettere, numeri e simboli). La scelta di questo approccio non è casuale, ma risponde a precise esigenze tecniche.

Non tutti i database gestiscono in maniera ottimale i file binari (BLOB). Utilizzando una rappresentazione testuale come Base64, è possibile archiviare i CV senza rischi di incompatibilità o limitazioni imposte dal DataBase.

Nel nostro caso, il CV viene recuperato tramite un tool del server MCP. Il fatto che sia già codificato in Base64 semplifica ulteriormente l'integrazione, in quanto il documento può essere gestito e trasferito come stringa testuale senza la necessità di ulteriori conversioni binarie.

La codifica Base64 garantisce che il documento sia rappresentato in un formato uniforme, indipendentemente dal sistema operativo o dal programma che l'ha generato. Questo riduce il rischio di errori durante i processi di importazione/esportazione o durante lo scambio tra piattaforme eterogenee.

```
_id: ObjectId('68a08107a7efbcc4a2edb403')
id : "82wBReruQDgcuFBXe7T6UyLWrdm2"
session : "ac0096b9-e2d3-4e66-a296-e5fedbd52126"
tipo : "application/pdf"
file : Binary.createFromBase64('JVBERi0xLjUKJb/3ov4KMIAwIG9iago8PCAvTGluZWYyaXplZCAxIC9MI0gzODk0IC9IIFsgMTIyMyAxOD...
uploadedAt : 2025-08-16T13:00:55.352+00:00
```

Figura IV.1: Collezione dei CV
Fonte: elaborazione degli autori

Sessions

In questa collezione avviene invece l'archiviazione di tutti i messaggi scambiati tra utente e IkiGAI, mantenendo informazioni come lo userID (id), la sessionID (session), un timestamp del caricamento, il percorso (path) (Capitolo II.0.1) e le domande con relative risposte.

```
_id: ObjectId('68b5b6774498f586dbfb816a')
id : "82wBReruQDgcuFBXe7T6UylWrdm2"
number_session : "649d6cec-dd23-40ea-b55e-c73f3268ed5f"
createdAt : 2025-09-01T15:06:31.400+00:00
path : "completed"
▼ q_and_a : Array (12)
  ▼ 0: Object
    question : "What activities make your heart sing and bring you joy?"
    answer : "coding"
    timestamp : 2025-09-01T15:06:31.400+00:00
  ▶ 1: Object
  ▶ 2: Object
  ▶ 3: Object
  ▶ 4: Object
  ▶ 5: Object
  ▶ 6: Object
  ▶ 7: Object
  ▶ 8: Object
  ▶ 9: Object
  ▶ 10: Object
  ▶ 11: Object
```

Figura IV.2: Collezione delle sessioni

Fonte: elaborazione degli autori

Questi dati vengono utilizzati per permettere all'utente di visualizzare le proprie vecchie sessioni in ogni momento.

IV.1.4. MCP [v1.13.0]

il protocollo MCP[8] è stato ampiamente descritto nel capitolo II.1. Nel progetto possiamo distinguere due specifici blocchi: "MCP HOST" e "MCP SERVER".

MCP Server

E' un programma che fornisce strumenti specifici e accesso a dati per gli LLM.[8] Sono presenti tre componenti: strumenti (**Tools**), risorse (**Resources**) e **Prompts**.

MCP Server tools

Per raggiungere lo scopo del progetto sono stati creati questi MCP tools:

- **analyze-cv-for-skill** - Analizza il CV dell'utente e restituisce una skill/professione principale.
- **search-jobs** - Ricerca lavori in base alle preferenze dell'utente.

- **save-pdf-to-mongo** - Salva un file PDF su MongoDB.
- **get-pdf-from-mongo** - Acquisisce un file PDF da MongoDB tramite lo userID e lo restituisce in formato base64.
- **save-session-data** - Salva una sessione composta da una domanda e una risposta su MongoDB.
- **get-session-data** - Acquisisce una sessione domanda e risposta da MongoDB tramite lo userID e il sessionID.
- **get-all-user-sessions** - Acquisisce tutte le sessioni di uno specifico utente da MongoDB.

MCP Host

Il componente MCP Host presenta tre sottocomponenti: l'interfaccia utente (**User Interface**), **LLM** e **MCP Client**. MCP Client è il ponte di collegamento tra il LLM e il MCP Server, ed è responsabile di inoltrare le richieste al Server e restituire il risultato.

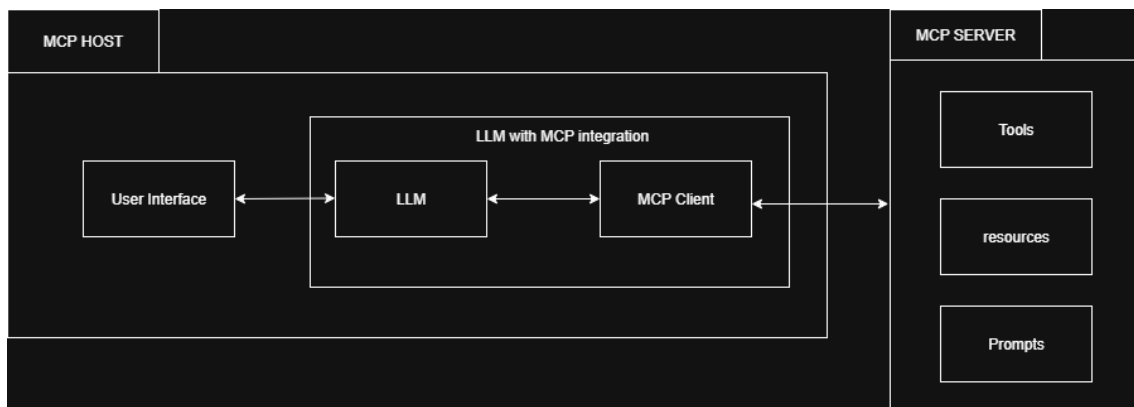


Figura IV.3: Concept Diagram

Fonte: adattato da Yijun, *Che cos'è MCP (Model Context Protocol) e come funziona*, Logto Blog, 2025.[8]

Nel progetto, la fase di profilazione, in cui vengono definite delle domande e ci si aspetta una risposta da parte dell'utente, è stata sviluppata con un approccio meno iterativo e più sequenziale nella gestione delle domande. Ciò significa che il flusso di interazione in questa fase segue una logica lineare, semplificando la raccolta delle informazioni essenziali.

Naturalmente, se durante questa fase l'utente richiede ulteriori chiarimenti sulla domanda corrente o pone una domanda di carattere più generale, il modello è comunque in grado di rispondere in modo adeguato. Questa scelta progettuale è stata adottata perché, nelle fasi iniziali, risultava la soluzione più semplice e immediata da implementare.

Successivamente, nella fase del career coach, è stato invece possibile introdurre una maggiore flessibilità nell'utilizzo degli LLM, fornendo indipendentemente le capacità dell'MCP, così da rendere il modello autonomo al 100%.

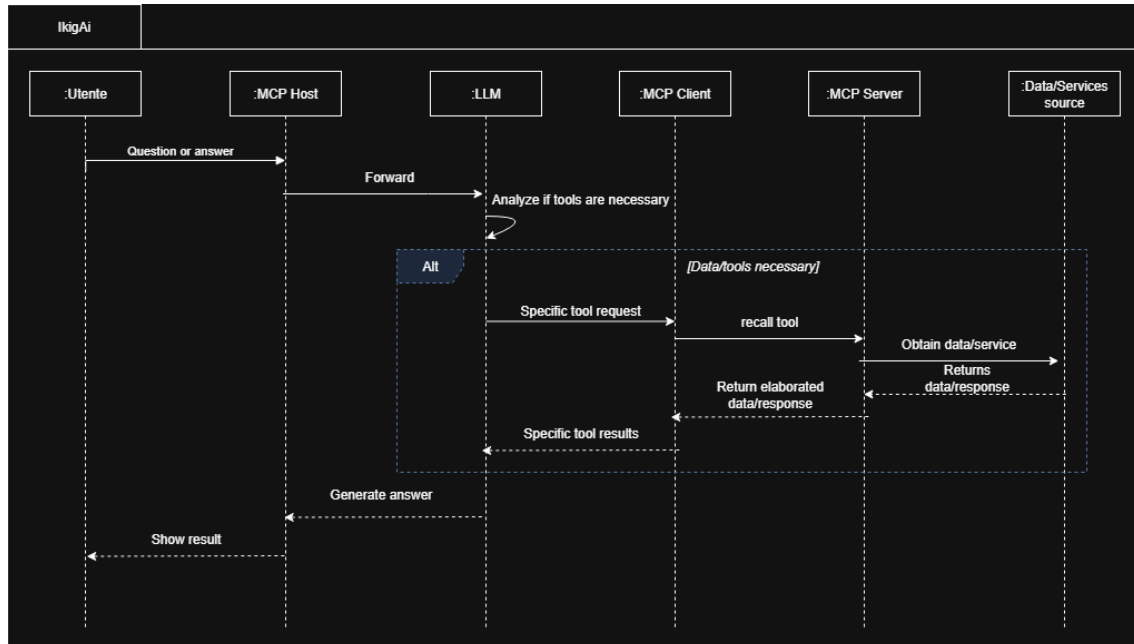


Figura IV.4: Sequence Diagram

Fonte: adattato da Yijun, *Che cos'è MCP (Model Context Protocol) e come funziona*, Logto Blog, 2025.[8]

Il diagramma (Fig.IV.4) illustra in modo dettagliato il flusso di interazione del sistema, evidenziando le dinamiche di comunicazione tra i diversi attori coinvolti. L'architettura si articola in più componenti; l'Utente, l'MCP Host, il modello linguistico di grandi dimensioni (LLM), l'MCP Client, l'MCP Server e infine le sorgenti dati o di servizio, queste collaborano per garantire l'elaborazione e la restituzione di una risposta completa ed efficace alla richiesta iniziale.

Il processo prende avvio con l'invio di una domanda o di un input da parte dell'utente. Tale richiesta, che rappresenta il punto di ingresso del flusso, viene immediatamente inoltrata all'MCP Host, il quale svolge la funzione di intermediario, trasferendo l'informazione all'LLM. In questa fase il modello ha il compito di analizzare la query ricevuta e di determinarne la natura. In particolare, il LLM deve stabilire se la richiesta può essere gestita autonomamente, attraverso le proprie capacità, oppure se sia necessario ricorrere a strumenti o servizi esterni per ottenere dati aggiuntivi o informazioni contestuali.

Se il modello ritiene di avere già tutte le informazioni necessarie, procede subito a generare una risposta. Al contrario, se capisce che servono dati aggiuntivi o funzioni esterne, attiva un flusso alternativo (ulteriormente se gli viene chiesto esplicitamente l'esecuzione di un tool, l'LLM procede con l'esecuzione di esso). In questo caso il LLM

prepara una richiesta e la invia all'MCP Client, che la passa all'MCP Server. Il server ha il compito di richiamare lo strumento giusto, cioè di contattare la fonte dati o il servizio esterno (Data/Services source) in grado di fornire le informazioni mancanti.

Quando un servizio esterno elabora una richiesta, invia al MCP Server la risposta con i dati necessari. Il server li rielabora e li passa al MCP Client, che li rende disponibili al modello linguistico. A questo punto, l'LLM utilizza queste informazioni insieme alle proprie per creare una risposta finale chiara, completa e arricchita dai dati provenienti dall'esterno.

Il flusso si conclude con l'invio della risposta, attraverso l'MCP Host, all'utente che aveva originato la richiesta.

IV.1.5. Adzuna [15]

E' un motore di ricerca che permette di trovare lavori in modo più intelligente e trasparente, e che aiuta ad evitare le migliaia offerte di lavoro irrilevanti.

L'inclusione nel progetto è data dal fatto che **Adzuna** offre una sua **API**[16] la quale permette di ottenere gli annunci e i dati più recenti in modo veloce ed efficace.

Nell'implementazione all'interno del nostro sistema, abbiamo deciso di basarci sui parametri principali che incidono maggiormente sulla ricerca di un'offerta di lavoro. Questi parametri vengono definiti dinamicamente sulla base delle risposte fornite dall'utente al bot:

- **country** - indica il paese nel quale si intende cercare lavoro (ad esempio: *it* per Italia, *uk* per Regno Unito). Questo parametro è fondamentale in quanto determina la base geografica della ricerca.
- **where** - specifica la città o la località desiderata per l'impiego (ad esempio: "Milano", "Roma").
- **full_time** / **part_time** - permette di filtrare la ricerca in base alla tipologia di contratto: a tempo pieno o a tempo parziale.
- **company** - consente di restringere i risultati a una particolare azienda, qualora l'utente fosse interessato a lavorare solo presso un determinato datore di lavoro.
- **salary_min** - definisce il salario minimo richiesto dall'utente, in modo da escludere automaticamente offerte che non rispettano le aspettative economiche dichiarate.

L'API offre ulteriori parametri che permettono di raffinare la ricerca (ad esempio per settore lavorativo, livello di esperienza, data di pubblicazione dell'annuncio, ecc.). Tuttavia, al fine di non complicare eccessivamente l'esperienza utente, abbiamo scelto

di limitare i campi ai soli parametri fondamentali. Qualora in futuro si volesse garantire un livello di personalizzazione maggiore, il tool MCP può essere facilmente esteso per includere ulteriori filtri.

È importante evidenziare che l'utilizzo di parametri multipli e troppo restrittivi può ridurre significativamente il numero di risultati ottenuti. Un esempio pratico si riscontra nel contesto italiano: la tipologia di contratto (part-time/full-time) e di salario minimo porta spesso ad avere pochissime offerte corrispondenti, se non addirittura nessuna. Ciò evidenzia una differenza rispetto a paesi con mercati del lavoro più ampi e trasparenti (come Regno Unito o Stati Uniti), dove i dati restituiti sono generalmente più numerosi e diversificati.

IV.1.6. Strumenti di versioning

Git[17] è un sistema di controllo delle versioni distribuito, gratuito e open source, progettato per gestire progetti di ogni tipo, dai più piccoli ai più grandi, con rapidità ed efficienza.

GitHub[18] è un sistema basato su git ma offre anche un'interfaccia utente accessibile comodamente da web.

La scelta di adottare Git è stata naturale: è uno strumento indispensabile perché permette a più sviluppatori di collaborare in modo efficace. Inoltre, essendo oggi uno standard diffuso nel mondo del lavoro, il suo utilizzo risulta coerente con le pratiche professionali attuali.

IV.1.7. Librerie

Zod [14] [v3.25.76]

E' una libreria di validazione basata su Typescript e viene utilizzata per definire schemi utilizzabili per convalidare i dati.

Caratteristiche:

- Funziona con Node.js e tutti i browser moderni.
- Conversione dello schema JSON integrata.
- API immutabile: i metodi restituiscono una nuova istanza.
- Nessuna dipendenza esterna.

Nel nostro progetto, Zod è stato impiegato per validare i dati passati dagli LLM o direttamente ai tool MCP. Questo ha permesso di garantire che le informazioni rispettassero sempre lo schema previsto, riducendo al minimo errori ed evitando l'elaborazione di valori

non validi. Inoltre, l'utilizzo di Zod si è rivelato utile anche durante le fasi di debug, facilitando l'individuazione di eventuali anomalie nei dati.

Pdf2json [19] [v3.2.0]

E' un modulo node.js che converte i PDF binari in JSON e testo. Sviluppato con pdf.js , estrae contenuti testuali ed elementi di form interattivi per l'elaborazione lato server e l'utilizzo da riga di comando.

Caratteristiche:

- Estrazione di testo PDF : estrae il contenuto testuale dei documenti PDF in JSON strutturato.
- Gestione degli elementi del modulo : analizza i campi del modulo interattivi nei PDF per un'acquisizione flessibile dei dati.
- Versatilità lato server e da riga di comando : integrazione con servizi Web per l'elaborazione remota di PDF o utilizzo come strumento da riga di comando autonomo per la conversione di file locali.
- Nessuna dipendenza : completamente privo di dipendenze dalla versione 3.1.6, solo codice JavaScript puro.

Nel progetto, Pdf2json è stato utilizzato per convertire i CV caricati in formato Base64 in testo, in modo da poterli successivamente elaborare e analizzare tramite gli LLM. Questo passaggio è stato fondamentale per rendere leggibile il contenuto del CV ai modelli.

React [20] [v19.0.0]

Consiste in una libreria che consente di creare interfacce utente partendo da singoli elementi chiamati componenti, i quali possono essere utilizzati e combinati in schermate, pagine web o app complete. Grazie alla struttura a componenti, React permette di organizzare il codice in modo modulare e riutilizzabile, facilitando manutenzione e scalabilità. E' possibile organizzare i componenti e il flusso logico dell'applicazione utilizzando due approcci principali: pages e app.[28]

L'approccio basato su **pages** prevede che ogni file all'interno della cartella pages corrisponda automaticamente a una rotta dell'applicazione. Ad esempio, il file pages/about.js sarà accessibile tramite l'URL /about. Questo modello semplifica la gestione del routing e permette di strutturare l'applicazione in modo lineare, associando chiaramente ciascuna pagina a una specifica rotta. Tuttavia, la gestione dei dati e delle API tende a essere

distribuita all'interno dei componenti della pagina, il che può risultare meno flessibile in applicazioni complesse o modulari.

L'approccio basato su **app**, invece, non lega i componenti direttamente alle rotte, ma organizza l'applicazione in moduli o pacchetti autonomi, ciascuno contenente componenti, logica e API dedicate. Questo permette di creare strutture più modulari e riutilizzabili, dove le API possono essere gestite in modo centralizzato e i componenti possono essere impiegati in diverse parti dell'interfaccia senza dipendere da una pagina specifica. Tale approccio favorisce inoltre un flusso di dati coerente tra frontend e backend, rendendo più semplice l'integrazione di più strumenti o servizi all'interno dello stesso progetto.

Nel progetto, l'adozione di un'architettura basata su app ha consentito una gestione più modulare dei componenti e delle API, migliorando la comunicazione tra i tool MCP e il frontend, senza essere vincolati alla struttura delle singole pagine.

React-icons [21] [v5.5.0]

E' una libreria molto utilizzata perché permette di importare facilmente migliaia di icone, utili a migliorare notevolmente la UI.

UUID [22] [v11.1.0]

Viene utilizzato per la creazione di identificatori unici universali (universally unique identifiers) RFC9562[23].

React Simple Typewriter [24] [v5.0.1]

E' un semplice componente react che aggiunge una animazione di scrittura ad uno specifico testo.

IV.2. Panoramica visuale del progetto

In questa figura possiamo osservare come si presenta la pagina principale del sito, dove viene spiegato il concetto chiave del progetto con in parte immagine rappresentativa del concetto di ikigai.

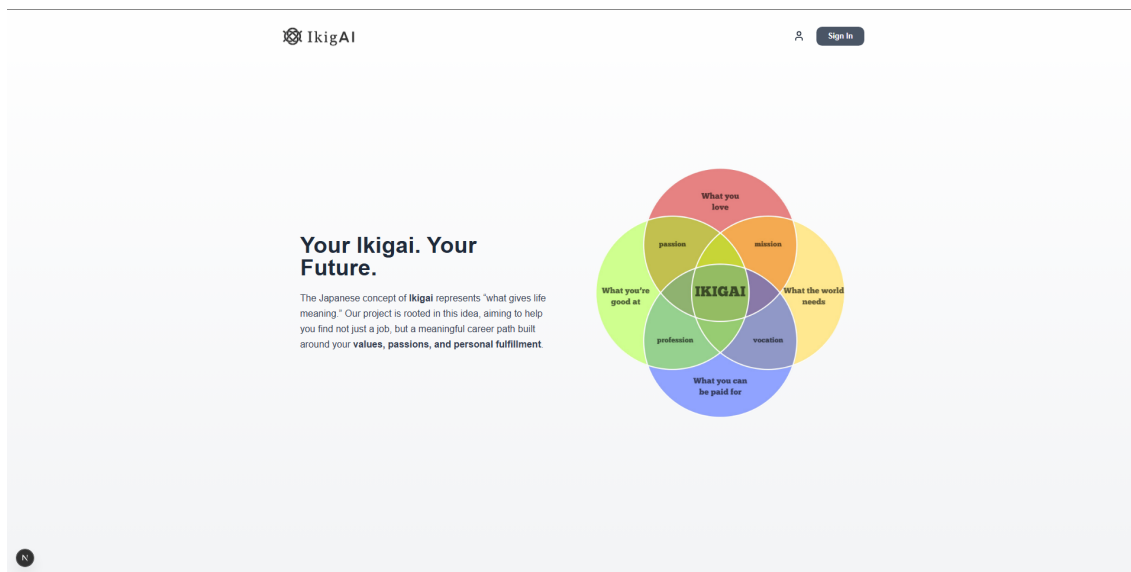


Figura IV.5: Initial Page
Fonte: elaborazione degli autori

Viene data anche una piccola descrizione delle funzionalità che troveremo una volta eseguito il sign in.

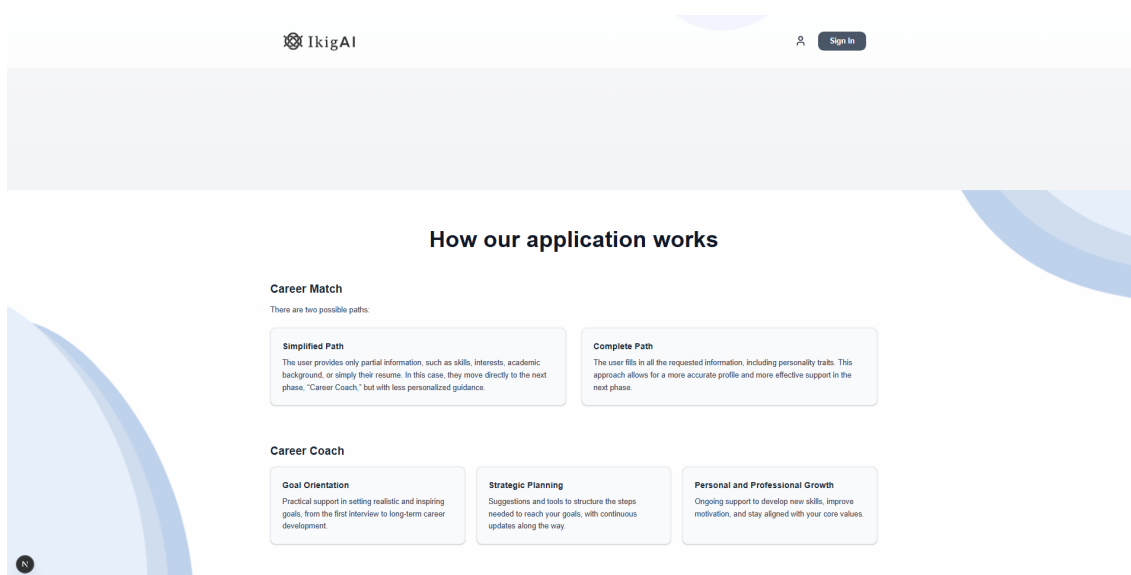


Figura IV.6: Initial Page
Fonte: elaborazione degli autori

Il login e le sessioni, come già menzionato, vengono gestite da Firebase. Una funzionalità in più che è stata aggiunta è quella di potersi direttamente loggare tramite un account google.

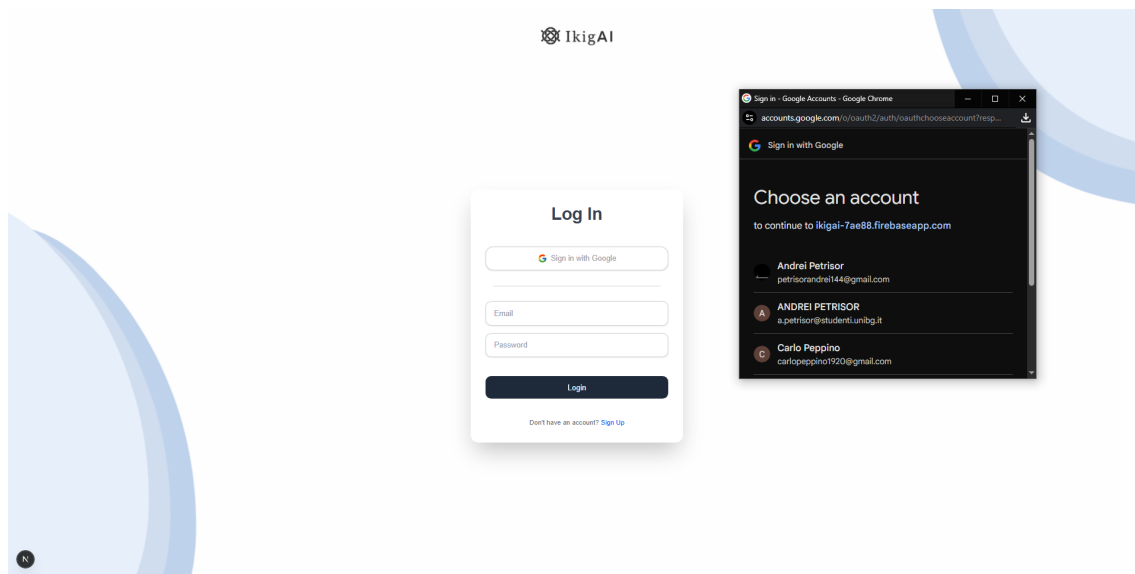


Figura IV.7: login
Fonte: elaborazione degli autori

Per scegliere il percorso desiderato basta cliccare su una delle due opzioni mostrate in questa vista. Si verrebbe indirizzati alla pagina scelta.

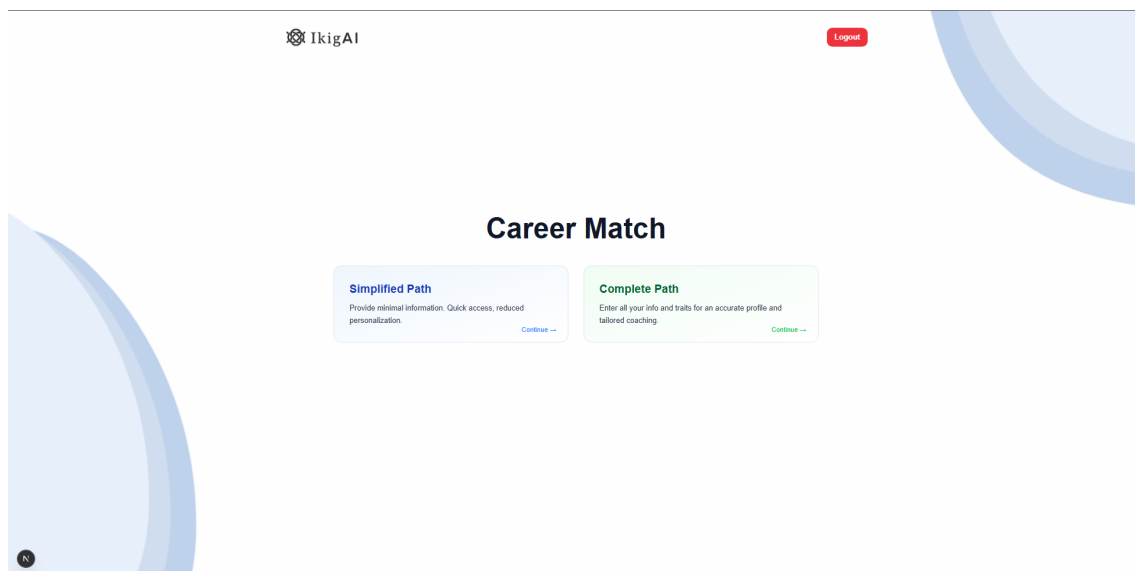


Figura IV.8: Choosing Path
Fonte: elaborazione degli autori

Nel Completed Path come già spiegato precedentemente nel cap. II.0.1, troviamo più domande incentrate sull'ikigai, per avere un pool di informazioni maggiore.

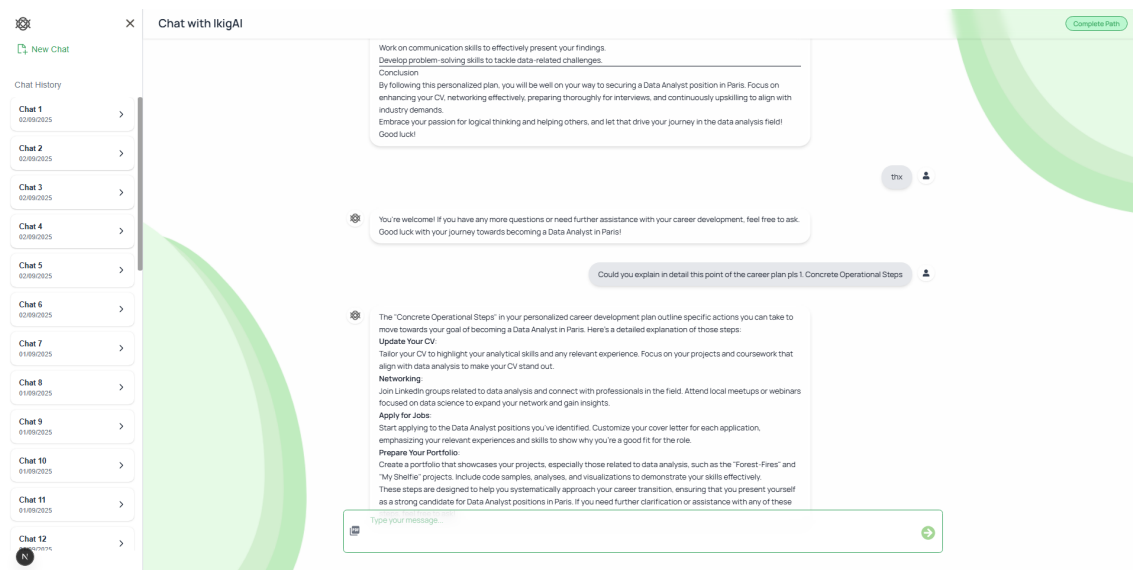


Figura IV.9: Completed Path
Fonte: elaborazione degli autori

Nel Simplified Path, invece, ci siamo limitati a chiedere poche domande per capire che professione potrebbe essere adeguata per l'utente. In questa parte è stato utilizzato il tool MCP per analizzare il CV(se inserito) e trovare la migliore professione in base alle informazioni presenti. Successivamente il percorso prosegue con il flusso standard, che prevede innanzitutto la visualizzazione delle offerte di lavoro localizzate nell'area specificata dall'utente e, in seguito, l'avvio della fase di career coaching.

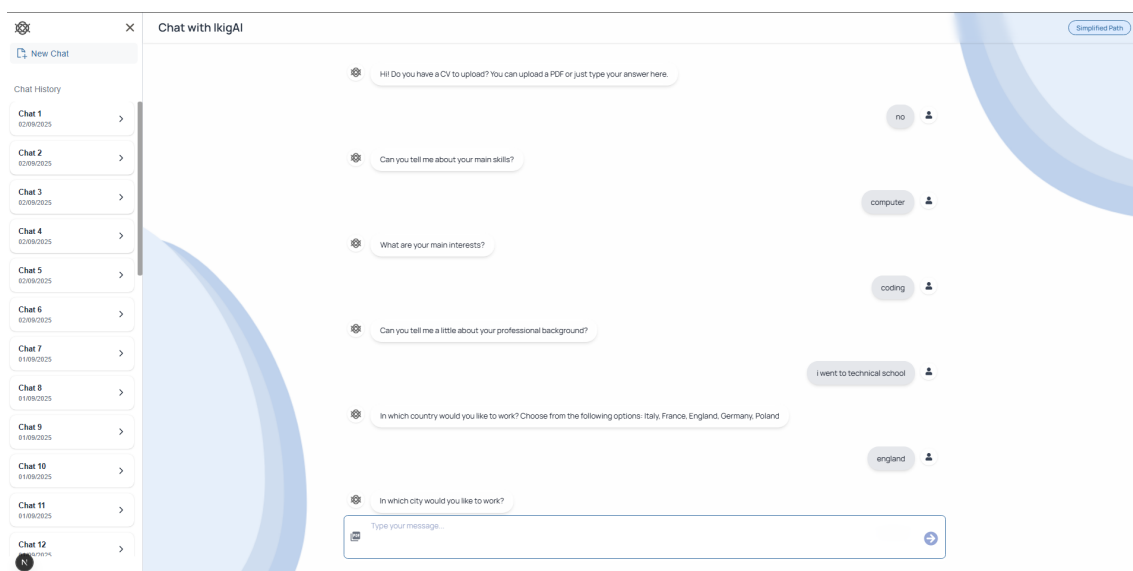


Figura IV.10: Simplified Path
Fonte: elaborazione degli autori

Per avere una visione completa del progetto è disponibile un video illustrativo su

youtube[25] oppure, in modo più completo ma più dispendioso e complesso, è disponibile il download del progetto da GitHub[26].

IV.3. Criticità riscontrate e strategie di risoluzione

Durante lo sviluppo del progetto sono emerse alcune sfide che hanno richiesto scelte tecniche complesse.

Una prima difficoltà ha riguardato la gestione delle sessioni conversazionali. In una fase iniziale si era pensato di avere una sola chat e di avere lo storico di chat che venivano rese disponibili all'LLM. In corso d'opera, tuttavia, l'architettura è stata ridefinita: si è deciso di rendere disponibile la cronologia delle conversazioni già svolte, ma con un rientro diretto nella fase di career coaching. In questo modo l'utente può comunque consultare lo storico dei messaggi e proseguire l'interazione da un contesto chiaro, pur senza ripristinare in senso stretto lo stato intermedio del flusso.

Un secondo aspetto critico ha riguardato l'integrazione tra Engine, LLM e MCP Server. L'adozione combinata di queste tecnologie, decisamente nuove per il team, ha richiesto tempo per comprendere le modalità ottimali di gestione: l'Engine come cuore logico, il modello GPT come generatore di risposte e il MCP come fornitore di tool e accesso a risorse esterne. Questa esperienza ha portato alla creazione di un'infrastruttura solida e flessibile, che può essere ampliata con facilità per supportare nuovi casi d'uso.

Un'ulteriore sfida si è manifestata sul piano prestazionale. La generazione dei piani personalizzati di career coaching poteva comportare tempi di attesa non trascurabili. La soluzione adottata è stata duplice: da un lato, l'utilizzo meticoloso di modelli diversi (GPT-4o-mini per attività più complesse, GPT-3.5 per quelle più leggere), dall'altro, la presentazione dei contenuti in stile chat interattiva. Questa scelta non solo ha ridotto i tempi percepiti, ma ha anche reso l'esperienza utente più naturale e fluida, simile a quella di un assistente conversazionale tradizionale.

Infine, la gestione della sicurezza ha rappresentato un punto di attenzione. Per evitare di implementare da zero meccanismi di autenticazione con il rischio di introdurre vulnerabilità, si è scelto di delegare la responsabilità a Firebase. Questo approccio ha permesso di garantire robustezza e scalabilità senza appesantire l'architettura interna.

IV.4. Costi

Un aspetto importante da considerare quando si valuta l'architettura riguarda i costi legati all'uso dei modelli di intelligenza artificiale. Servizi come Firebase o MongoDB hanno spesso piani gratuiti o costi contenuti, che dipendono dallo spazio usato o dal numero di

richieste. Con i modelli linguistici di grandi dimensioni, come GPT-3.5 o GPT-4, la logica è diversa: ogni utilizzo ha un costo diretto, calcolato in base ai token elaborati.

I token non sono altro che piccoli pezzi di testo, in pratica frammenti di parole. Nel calcolo dei costi di un modello come GPT-3.5 o GPT-4o mini:, vengono considerati sia i token inviati al modello, cioè quelli che l'Engine manda tramite la libreria OpenAI, sia i token generati dal modello restituiti come risposta. In pratica, ogni interazione con il chatbot, dalle prime domande di profilazione fino alla creazione del piano di career coaching, contribuisce al costo in base alla quantità totale di testo scambiata.

IV.4.1. Differenza GPT-3.5 e GPT-4o mini:

Il modello GPT-3.5 è stato scelto come motore principale per le interazioni più frequenti, poiché offre un buon compromesso tra qualità delle risposte e costo per token. Laddove, invece, è stato necessario ottenere risposte di livello superiore in termini di coerenza, profondità e capacità di ragionamento, è stato integrato anche GPT-4, che però ha un costo significativamente più elevato.

Costi per token dei modelli OpenAI:

- GPT-3.5 Turbo: \$0.0005 per 1.000 token in input e \$0.0015 per 1.000 token in output.
- GPT-4o mini: \$0.60 per 1.000 token in input e \$2.40 per 1.000 token in output.

Nel caso del nostro sistema, una sessione completa, dalla profilazione fino alla generazione del piano di career coaching, utilizza così pochi token che il costo totale rimane inferiore a 0,01 dollari. Questo dimostra come sia possibile offrire un servizio di coaching personalizzato in modo estremamente economico. Se si prosegue con la chat il costo aumenta fino a raggiungere pochi centesimi.

IV.4.2. Prospettiva futura

L'aspetto economico resta un fattore chiave per l'evoluzione del progetto. In prospettiva, l'architettura potrà integrare ulteriori modelli alternativi, anche open source, che permettano di ridurre i costi mantenendo un livello qualitativo adeguato. Inoltre, si potrà sfruttare il continuo aggiornamento dell'offerta di OpenAI e di altri provider, che stanno introducendo modelli sempre più performanti a costi inferiori.

L'equilibrio tra GPT-3.5 per le interazioni standard e GPT-4o mini: per i compiti più complessi consente di mantenere la qualità del servizio senza comprometterne la sostenibilità economica.

V. Conclusioni

V.1. Sintesi dei risultati ottenuti

Il prototipo sviluppato consiste in un chatbot basato su un modello di linguaggio di grandi dimensioni, integrato in una piattaforma web accessibile pubblicamente e gratuitamente. Il modello, durante la fase di profilazione, è in grado di raccogliere informazioni dall'utente e fornire suggerimenti di career match con risultati soddisfacenti, anche senza un fine-tuning specifico mirato al task. Per quanto riguarda il career coaching, il sistema si dimostra efficace nel fornire consigli pratici e personalizzati, supportando l'utente nella definizione di percorsi professionali coerenti con il proprio profilo.

L'integrazione tra LLM, server MCP e strumenti aggiuntivi mostra chiaramente ampi margini di miglioramento. Alcune funzionalità, come la gestione delle risorse disponibili sul server, sono ancora limitate, mentre i tool creati offrono già un buon supporto operativo. Tuttavia, vi è un significativo spazio per espandere le risorse, migliorare la consapevolezza del modello e introdurre nuove features per aumentare la qualità e l'affidabilità delle risposte generate.

In conclusione, il progetto rappresenta un risultato importante sia dal punto di vista tecnico sia da quello sperimentale. Nonostante le sfide iniziali e i limiti legati alla novità delle tecnologie utilizzate, è stato possibile sviluppare un prototipo funzionante, in grado di soddisfare le principali richieste del sistema di career coaching.

V.2. Sviluppi futuri

Il progetto realizzato rappresenta la prima esperienza per i singoli componenti, nello sviluppo di una piattaforma che integra modelli di intelligenza artificiale avanzati all'interno di un contesto applicativo web. Pur essendo la prima implementazione di questo tipo, i risultati ottenuti sono soddisfacenti, soprattutto considerando la complessità e la novità delle tecnologie coinvolte.

- La fase di profilazione, in particolare, è stata sviluppata seguendo un approccio sequenziale: le domande vengono presentate in ordine e ci si aspetta una risposta

specifica dall'utente. Questa scelta, meno iterativa rispetto a un modello conversazionale completamente dinamico, ha consentito di strutturare inizialmente il flusso in maniera chiara e gestibile. Tuttavia, il modello è comunque in grado di fornire chiarimenti o risposte aggiuntive se l'utente formula domande durante questa fase. Nelle fasi successive, come quella del career coaching, il sistema ha invece permesso un maggior livello di flessibilità, grazie alla gestione dinamica delle capabilities del server MCP rese disponibili al LLM, permettendo così di adattare le interazioni alle esigenze dell'utente in maniera più articolata.

- Un aspetto importante da considerare riguarda i costi operativi legati all'utilizzo dei modelli di linguaggio. In futuro, sarà fondamentale condurre un'analisi più approfondita, valutando quanto incide ciascuna sessione utente sul costo complessivo, la durata media delle interazioni e la frequenza di utilizzo da parte degli utenti più affezionati. Tali informazioni permetteranno di ottimizzare il sistema sia dal punto di vista economico sia da quello dell'efficienza del servizio.
- Dal punto di vista tecnico, l'implementazione delle nuove tecnologie, LLM, MCP, Next.js, Firebase e TypeScript, ha rappresentato una sfida significativa. L'apprendimento e l'integrazione di queste tecnologie non sono stati immediati, e i tempi di sviluppo sono stati influenzati anche dalla necessità di rispettare le tempistiche della metodologia Scrum adottata. Nonostante ciò, come detto precedentemente, il risultato complessivo del chatbot risulta buono. Con maggiore esperienza e tempi più ampi, il progetto potrebbe essere ulteriormente raffinato, migliorando la qualità ingegneristica e l'integrazione dei componenti, con l'obiettivo di trasformarlo in un prodotto completo e stabile.
- Un'altra possibile evoluzione riguarda la ripresa delle chat. Attualmente, se una sessione si interrompe durante la fase di profilazione, l'utente non può continuare esattamente dallo stesso punto; la sessione successiva riparte direttamente dalla fase di career coaching. In futuro, si prevede di implementare questa funzionalità, in modo da garantire una user experience più completa e fluida, permettendo di riprendere la conversazione esattamente dove era stata interrotta.
- Dal punto di vista tecnologico, il progetto ha evidenziato la possibilità di integrare ulteriori strumenti e risorse all'interno del server MCP, rendendo i moduli più consapevoli e capaci di supportare meglio il modello linguistico. L'adozione di nuove tecnologie o lo sviluppo di strumenti proprietari permetterebbe di ampliare le funzionalità disponibili e di rendere il sistema più completo e versatile.

Attualmente, il progetto è eseguito in locale, il che ha permesso al team di testare tutte le funzionalità in un ambiente controllato, riducendo eventuali problemi legati a infrastrutture esterne e garantendo maggiore flessibilità durante le fasi di sviluppo e sperimentazione. In futuro, uno degli obiettivi principali sarà la migrazione della piattaforma su un server dedicato, in modo da rendere il sistema accessibile a un numero più ampio di utenti in modo continuativo.

Questa transizione consentirà non solo di aumentare la disponibilità della piattaforma, ma anche di raccogliere dati concreti sull'utilizzo reale del sistema. Sarà possibile monitorare il traffico degli utenti, la durata e la frequenza delle sessioni, e comprendere meglio come i modelli di intelligenza artificiale vengono effettivamente utilizzati. Queste informazioni saranno fondamentali per valutare l'impatto economico dell'utilizzo dei LLM, stimare i costi associati a ciascuna sessione e identificare possibili strategie di ottimizzazione per rendere il servizio sostenibile nel tempo.

Per concludere, la futura evoluzione della piattaforma dovrà considerare anche aspetti legati alla sicurezza dei dati, alla gestione dei permessi e alla protezione della privacy degli utenti, soprattutto in un contesto in cui le informazioni personali e professionali vengono raccolte e analizzate. L'obiettivo sarà sviluppare un sistema completo, sicuro e scalabile, capace di integrare le nuove tecnologie emergenti e di offrire un servizio stabile, funzionale e sostenibile nel tempo.

Bibliografia

- [1] Sharma M., Kumari A., Jyotsna, *AI-Based Deep Learning Chatbot for Career and Personal Mentorship*, IEEE, 2020.
- [2] Jain K., Ms. P. Sravanthi, *Smart Chatbot to Assist Users of Employment Websites in Job Search, Skill Development, and Networking Opportunities*, IJRASET, 2024.
- [3] Krishnan A., Joseph J., Nihal N N, Fathima S S., *Skill Mount: Personalized Career Skills Development Using Machine Learning Algorithms*, IEEE, 2024.
- [4] Prakash S., Sundar K., Manohar E., Prakash L E., *Career Connect – A Personalized Job Recommendation AI Chatbot*, SSRN, 2024.
- [5] Shilaskar S., Bhatlawande S., Sawle P., Gupta S., Buche R. *Conversational AI for Career Counseling*, IEEE, 2024.
- [6] Arshad T K, Mohammed D., Arjun A, Anshad M, Jaseena C A, *VocaVisionary: A Career Guidance Chatbot*, IEEE, 2024.
- [7] Claude, *MCP*, url: <https://modelcontextprotocol.io/docs/getting-started/intro>.
- [8] Yijun, *Che cos'è MCP (Model Context Protocol) e come funziona*, url: <https://blog.logto.io/it/what-is-mcp>, 2025.
- [9] Atlassian, *Scrum*, url: <https://www.atlassian.com/it/agile/scrum>.
- [10] Robert Cecil Martin , *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Addison-Wesley , 2017.
- [11] NextJs, *NextJs*, url: <https://nextjs.org/>
- [12] Firebase, *Firebase*, url: <https://firebase.google.com/docs/auth?hl=it>.
- [13] MongoDB Atlas, *MongoDB Atlas*, url: <https://www.mongodb.com/docs/atlas/>
- [14] Zod, *Zod*, url: <https://zod.dev/>

- [15] Adzuna, *Adzuna*, url: <https://www.adzuna.it/about-us.html>
- [16] Adzuna, *Adzuna API*, url: <https://developer.adzuna.com/>
- [17] Git, *Git*, url: <https://git-scm.com/>
- [18] GitHub, *GitHub*, url: <https://github.com/>
- [19] Pdf2json, *npm*, url: <https://www.npmjs.com/package/pdf2json>
- [20] React, *React*, url: <https://react.dev/>
- [21] React-icons, *React-icons*, url: <https://react-icons.github.io/react-icons/>
- [22] uuid, *npm*, url: <https://www.npmjs.com/package/uuid>
- [23] RFC9562, *RFC*, url: <https://www.rfc-editor.org/rfc/rfc9562.html>
- [24] react-simple-typewriter, *npm*, url: <https://www.npmjs.com/package/react-simple-typewriter>
- [25] Andrei Petrisor, *IkigAI video*, url: <https://www.youtube.com/watch?v=RByfDj5VZm8>
- [26] Radu Antonio Valentin e Petrisor Andrei, *IkigAI*, url: <https://github.com/petri2212/IkigAI>.
- [27] Wikipedia, *Base64*, url: <https://en.wikipedia.org/wiki/Base64>
- [28] Tanzim Chowdhury, *Next.js: App Router vs Pages Router — What You Need to Know (2024)*, url: <https://medium.com/@tanzim3421/next-js-app-router-vs-pages-router-what-you-need-to-know-202-69a885ccaa56>
- [29] ByteByteGo, *Why is everyone talking about MCP?*, url: https://www.youtube.com/watch?v=_d0duu3dED4, 2025.
- [30] CareerVillage.org, *COACH: AI-powered Career Coach | CareerVillage*, url: <https://www.aicareercoach.org/>.
- [31] Himmalayas, *AI Career Coach*, url: <https://himalayas.app/ai-career-coach>.